SHREE L.R TIWARI DEGREE COLLEGE OF ARTS SCIENCE AND COMEERECE

Certificate

Name: . Class :

Roll No.: Exam No. :

Institution _____

This is certified to be the bonafide work of the student in the

_____ *Laboratory during the academic year 20 / 20 .*

No. of practicals certified _____ *out of* _____ *in the subject of* _____

.....

Txacher In-charge

....

Examiner's Signature *Principal*

Date :..... *Institution Rubber Stamp*

(N.B : The candidate is expected to retain his/her journal till he/she passes in the subject.)

SHREE L.R TIWARI DEGREE COLLEGE OF ARTS SCIENCE AND COMEERECE

IOT PRACTICAL BOOK

Sr. No.	Practical Title	Date	Page No.	Signature
1	**LED Blinking Patterns using GPIO**			
2	**Telegram GPIO Control**			
3	**TM1637 Time Display**			
4	**GPS Module Interface**			
5	**ADS1115 Oscilloscope**			
6	**16×2 LCD using I²C**			
7	**RFID Read/Write Interface**			
8	**Fingerprint Sensor Interface**			
9	**Pi Camera Image & Video Capture**			
10	**IoT Home Automation using Raspberry Pi**			

Raspberry Pi 4 B J8 GPIO Header

Pin#	NAME		NAME	Pin#
01	3.3v DC Power		DC Power 5v	02
03	GPIO02 (SDA1, I²C)		DC Power 5v	04
05	GPIO03 (SCL1, I²C)		Ground	06
07	GPIO04 (GPCLK0)		(TXD0, UART) GPIO14	08
09	Ground		(RXD0, UART) GPIO15	10
11	GPIO17		(PWM0) GPIO18	12
13	GPIO27		Ground	14
15	GPIO22		GPIO23	16
17	3.3v DC Power		GPIO24	18
19	GPIO10 (SPI0_MOSI)		Ground	20
21	GPIO09 (SPI0_MISO)		GPIO25	22
23	GPIO11 (SPI0_CLK)		(SPI0_CE0_N) GPIO08	24
25	Ground		(SPI0_CE1_N) GPIO07	26
27	GPIO00 (SDA0, I²C)		(SCL0, I²C) GPIO01	28
29	GPIO05		Ground	30
31	GPIO06		(PWM0) GPIO12	32
33	GPIO13 (PWM1)		Ground	34
35	GPIO19		GPIO16	36
37	GPIO26		GPIO20	38
39	Ground		GPIO21	40

Raspberry Pi 4 B J14 PoE Header

01	TR01		TR00	02
03	TR03		TR02	04

Pinout Grouping Legend

Inter-Integrated Circuit Serial Bus

Serial Peripheral Interface Bus

Ungrouped/Un-Allocated GPIO

Universal Asynchronous Receiver-Transmitter

Reserved for EEPROM

Rev. 2
19/06/2019 CGS

www.element14.com/RaspberryPi

IOT ALL PRCATICAL MANUAL

PRACTICAL 1:

AIM: CREATE DIFFERENT BLINKING PATTERNS WITH LEDS BY CONNECTING TO GPIO PINTS OF RASPBERRY PI.

COMPONENTS REQUIRED:

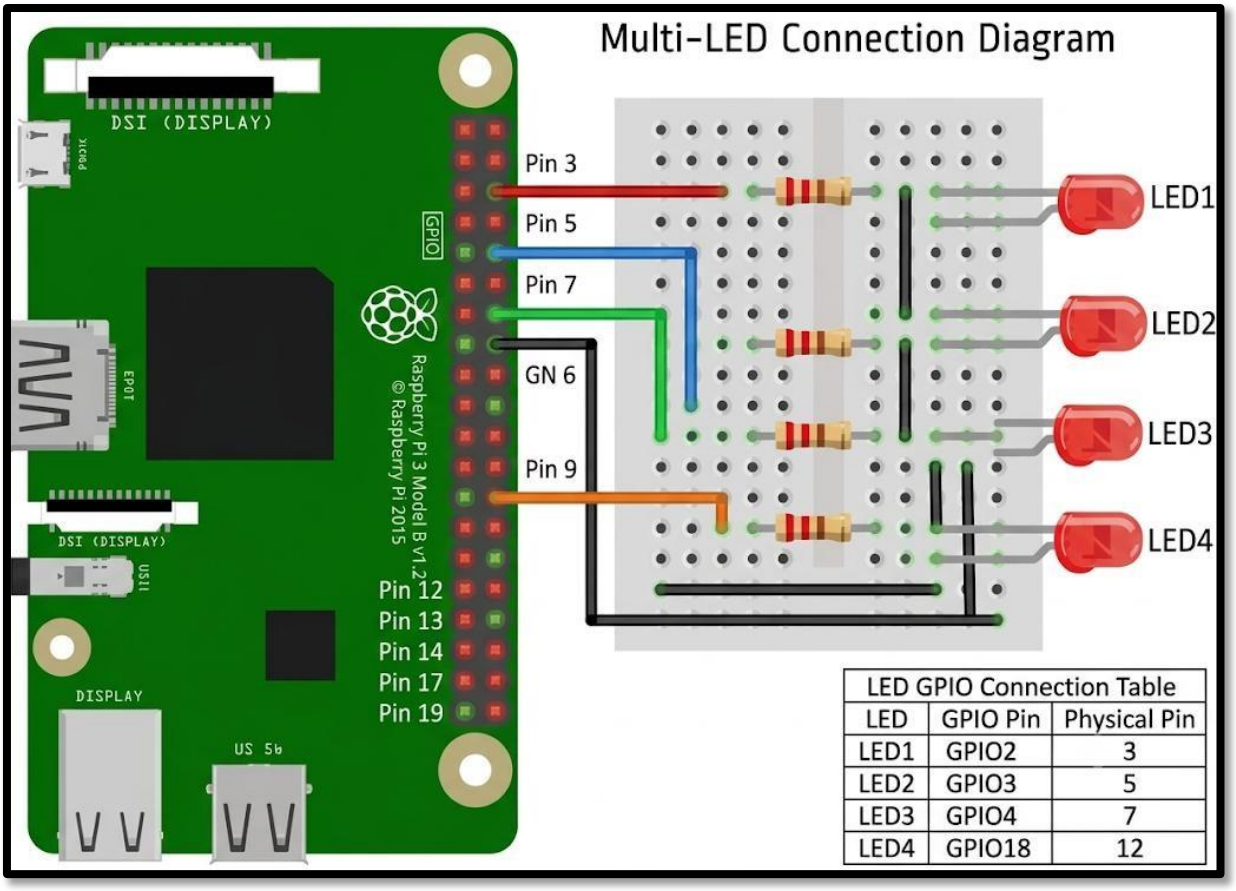
Sr. No.	Component	Quantity
1	**Raspberry Pi 4 B**	1
2	**Micro Sd Card for Raspberry Pi OS**	1
3	**Power Supply for Raspberry Pi**	1
4	**USB mouse cable**	1
5	**USB keyboard cable**	1
6	**RJ45 ethernet cable**	1
7	**HDMI cable**	1
8	**LED**	4
9	**Resistor**	4
10	**Breadboard**	1
11	**Jumper Wires**	As Required

CONNECTION PROCEDURE:

1. Power off the Raspberry Pi.
2. Place four LEDs on the breadboard.
3. Connect each LED anode (+) to the GPIO pins through a 220 Ω resistor:
    - LED1 → GPIO2 (Physical Pin 3)
    - LED2 → GPIO3 (Physical Pin 5)
    - LED3 → GPIO4 (Physical Pin 7)
    - LED4 → GPIO18 (Physical Pin 12)
4. Connect all LED cathodes (-) to any GND pin of the Raspberry Pi.
5. Power on the Raspberry Pi.
6. Open the Python IDE (Thonny) or Terminal.
7. Save the program and execute it.

8. Observe the running LED pattern.

CIRCUIT DIAGRAM:



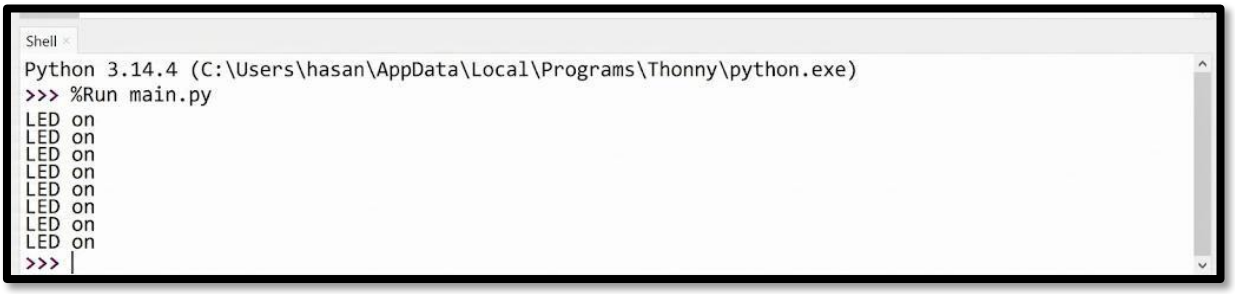
PYTHON CODE:

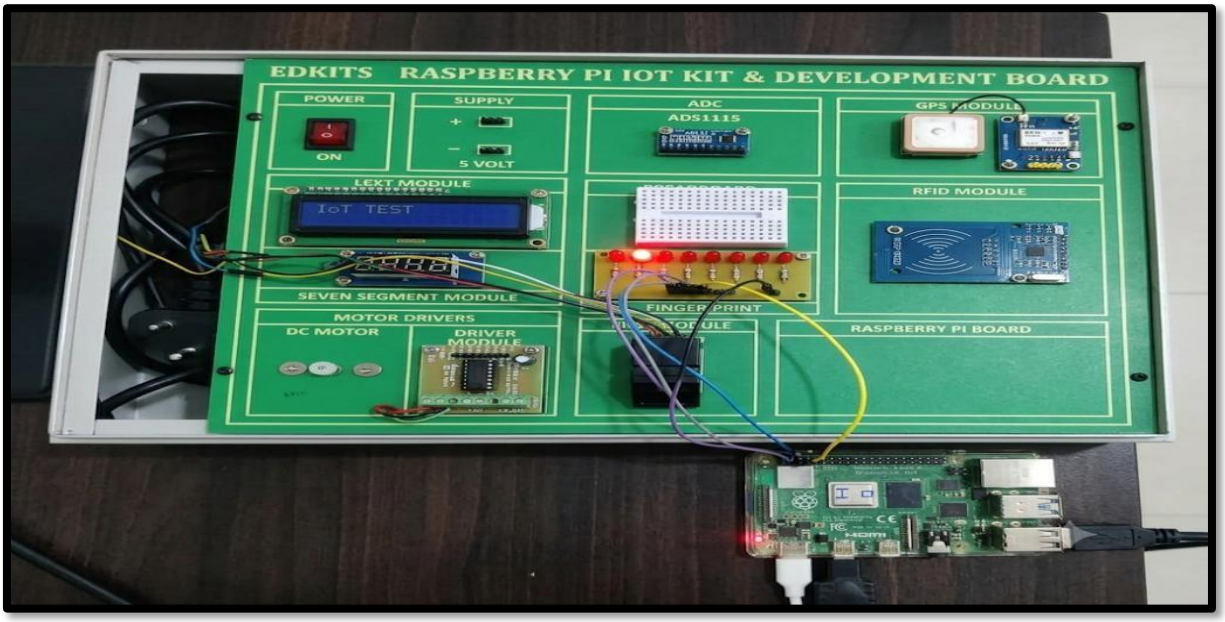
```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(2,GPIO.OUT)
GPIO.setup(3,GPIO.OUT)
GPIO.setup(4,GPIO.OUT)
GPIO.setup(18,GPIO.OUT)
while 1:
    time.sleep(.2)
    print("LED on")
    GPIO.output(2,GPIO.HIGH)
```

```
GPIO.output(3,GPIO.LOW)
GPIO.output(4,GPIO.LOW)
GPIO.output(18,GPIO.LOW)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.LOW)
GPIO.output(3,GPIO.HIGH)
GPIO.output(4,GPIO.LOW)
GPIO.output(18,GPIO.LOW)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.LOW)
GPIO.output(3,GPIO.LOW)
GPIO.output(4,GPIO.HIGH)
GPIO.output(18,GPIO.LOW)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.LOW)
GPIO.output(3,GPIO.LOW)
GPIO.output(4,GPIO.LOW)
GPIO.output(18,GPIO.HIGH)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.LOW)
GPIO.output(3,GPIO.LOW)
GPIO.output(4,GPIO.LOW)
GPIO.output(18,GPIO.HIGH)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.LOW)
GPIO.output(3,GPIO.LOW)
GPIO.output(4,GPIO.HIGH)
```

```
GPIO.output(18,GPIO.LOW)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.LOW)
GPIO.output(3,GPIO.HIGH)
GPIO.output(4,GPIO.LOW)
GPIO.output(18,GPIO.LOW)
time.sleep(.2)
print("LED on")
GPIO.output(2,GPIO.HIGH)
GPIO.output(3,GPIO.LOW)
GPIO.output(4,GPIO.LOW)
GPIO.output(18,GPIO.LOW)
```

OUTPUT:





PRACTICAL 2

AIM: USE TELEGRAM TO CONTROL RASPBERRY PI GPIO.

COMPONENTS REQUIRED:

Sr. No.	Component	Quantity
1	**Raspberry Pi 4 B**	1
2	**Micro Sd Card for Raspberry Pi OS**	1
3	**Power Supply for Raspberry Pi**	1
4	**USB mouse cable**	1
5	**USB keyboard cable**	1
6	**RJ45 ethernet cable**	1
7	**HDMI cable**	1
8	**LED**	4
9	**Resistor**	4
10	**Breadboard**	1
11	**Jumper Wires**	As Required

SOFTWARE REQUIRED

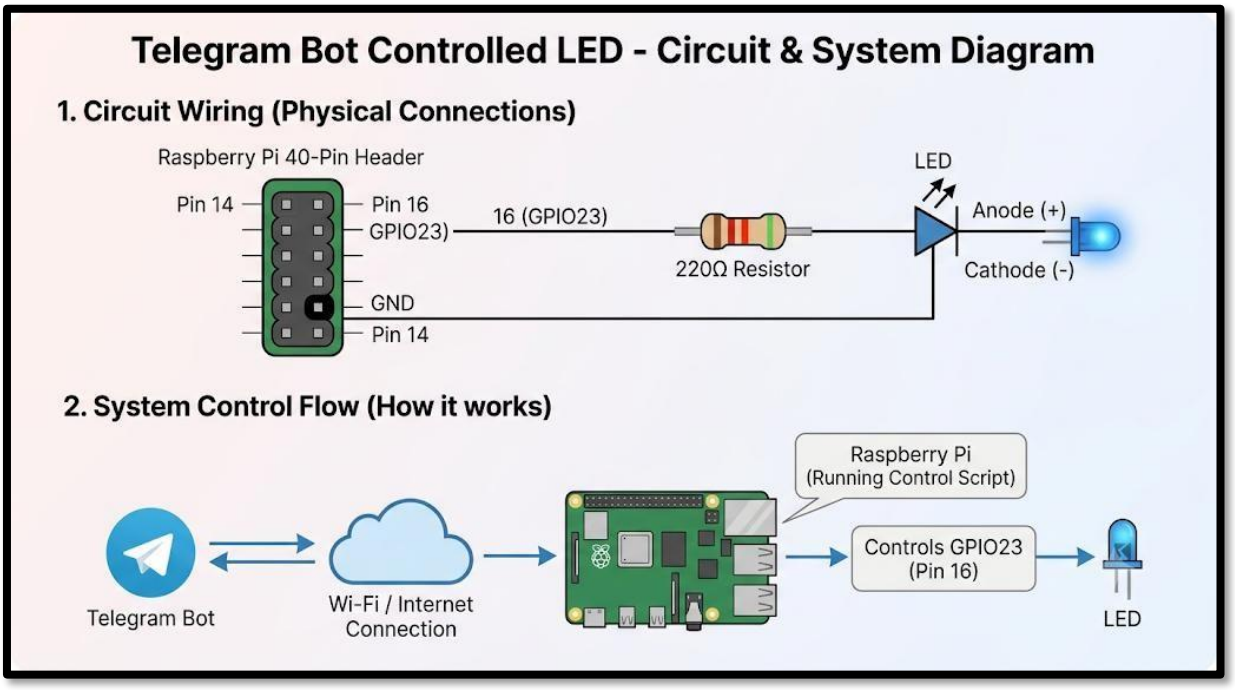
- Raspberry Pi OS
- Python 3
- Telegram App
- BotFather (to create Telegram Bot)
- python-telegram-bot library
- RPi.GPIO library

CONNECTION PROCEDURE:

1. Connect the LED anode (+) to **Physical Pin 16 (GPIO23)** through a **220 Ω resistor**.
2. Connect the LED cathode (-) to any **GND** pin.
3. Power ON the Raspberry Pi.
4. Install the required Python libraries.
5. Create a Telegram Bot using BotFather.
6. Copy the Bot Token.

7. Write the Python program.
8. Execute the program.
9. Open Telegram and send:
    a. **on**
    b. **off**
10. Observe the LED status.

CIRCUIT DIAGRAM:



INSTALLATION COMMANDS:

sudo apt update

sudo apt install python3-pip

pip3 install python-telegram-bot

sudo apt install python3-rpi.gpio

CREATE TELEGRAM BOT

1. Open Telegram.
2. Search **BotFather**.
3. Send

    /newbot

4. Enter Bot Name.

5. Enter Bot Username.
6. Copy the **Bot Token** generated by BotFather.

PYTHON CODE:

```
import sys
import time
import telepot
import RPi.GPIO as GPIO

# LED functions
def on(pin):
    GPIO.output(pin, GPIO.HIGH)
    return "LED ON"
def off(pin):
    GPIO.output(pin, GPIO.LOW)
    return "LED OFF"

# To use Raspberry Pi board pin numbers
GPIO.setmode(GPIO.BOARD)

# Set up GPIO output channel
GPIO.setup(16, GPIO.OUT)

def handle(msg):
    chat_id = msg['chat']['id']
    command = msg['text']

    print('Got command: %s' % command)

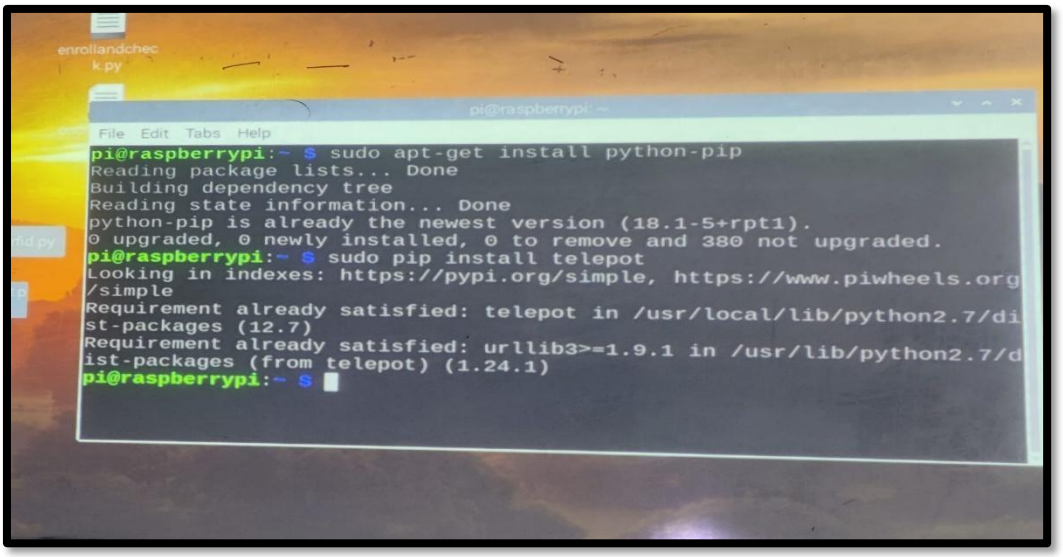
    if command == 'on':
```

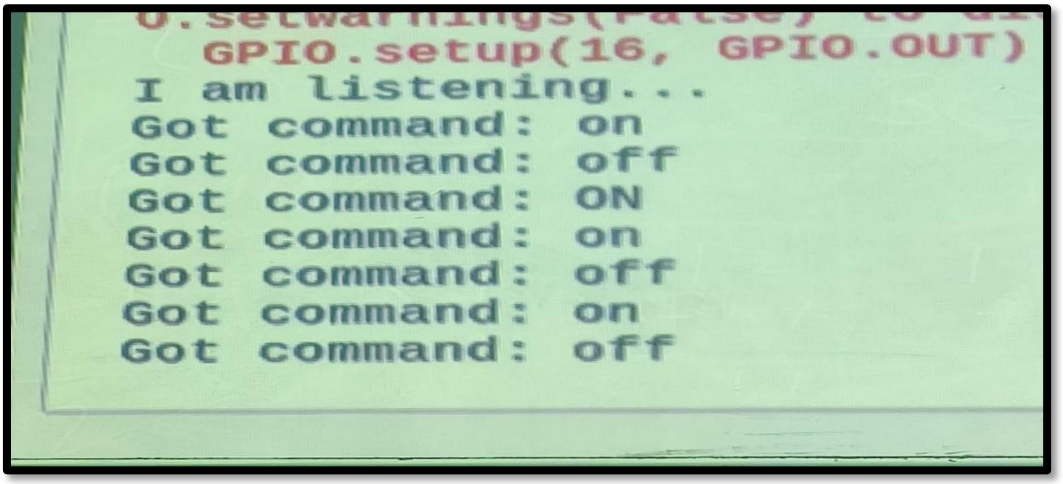
```
        bot.sendMessage(chat_id, on(16))
    elif command == 'off':
        bot.sendMessage(chat_id, off(16))

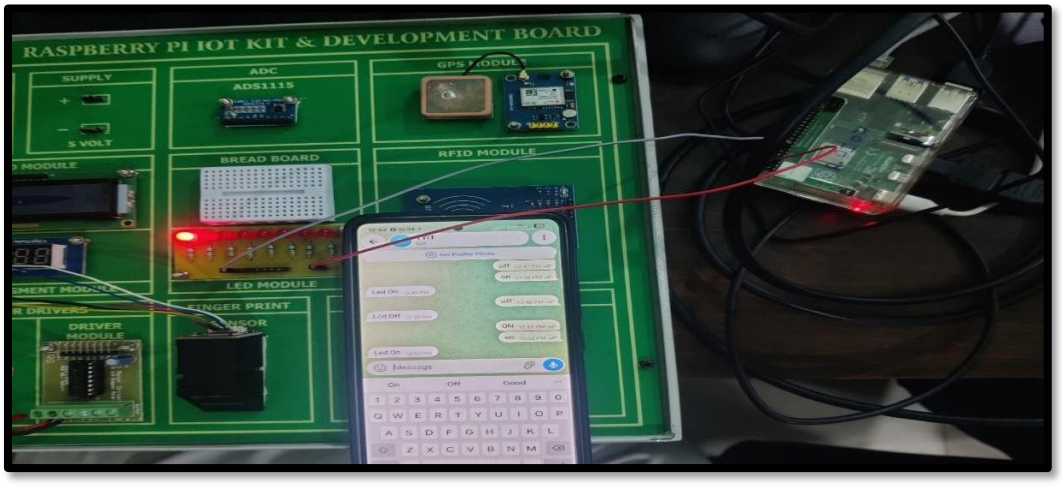

bot = telepot.Bot('8635706356:AAG3c9B0cB0B-LCHGeCXvlm4HASQhzM0aFA')
bot.message_loop(handle)
print('I am listening...')


while 1:
    try:
        time.sleep(1)
    except KeyboardInterrupt:
        print('\n Program interrupted')
        GPIO.cleanup()
        exit()
    except:
        print('Other error or exception occurred!')
        GPIO.cleanup()
```

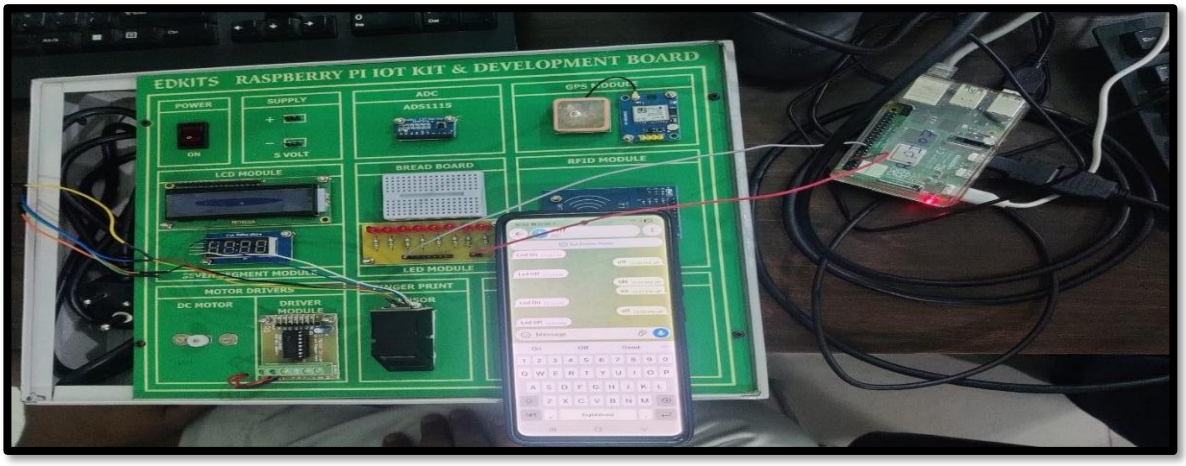
OUTPUT:


enrollandchec
k.py
pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~ $ sudo apt-get install python-pip
Reading package lists... Done
Building dependency tree
Reading state information... Done
python-pip is already the newest version (18.1-5+rpt1).
0 upgraded, 0 newly installed, 0 to remove and 380 not upgraded.
pi@raspberrypi:~ $ sudo pip install telepot
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org
/simple
Requirement already satisfied: telepot in /usr/local/lib/python2.7/di
st-packages (12.7)
Requirement already satisfied: urllib3>=1.9.1 in /usr/lib/python2.7/d
ist-packages (from telepot) (1.24.1)
pi@raspberrypi:~ $


GPIO.setup(16, GPIO.OUT)
I am listening...
Got command: on
Got command: off
Got command: ON
Got command: on
Got command: off
Got command: on
Got command: off


RASPBERRY PI IOT KIT & DEVELOPMENT BOARD
SUPPLY
ADC
ADS1115
BREAD BOARD
RFID MODULE
LED MODULE
DRIVER MODULE

4


EDKITS RASPBERRY PI IOT KIT & DEVELOPMENT BOARD
POWER
ON
SUPPLY
5 VOLT
ADC
ADS1115
GPS MODULE
LCD MODULE
BREAD BOARD
RFID MODULE
LED MODULE
MOTOR DRIVERS
DC MOTOR
DRIVER MODULE

PRACTICAL 3

AIM: INTERFACE TM1637 WITH RASPBERRY PI AND DISPLAY CURRENT TIME

REQUIREMENTS:

1. Raspberry Pi
2. TM1637 4-digit 7-segment display
3. Jumper wires
4. Raspberry Pi power supply
5. MicroSD card with Raspberry Pi OS

COMPONENTS / PIN CONFIGURATION:

TM1637 Pin	Raspberry Pi Pin
VCC	5V (Pin 2)
GND	GND (Pin 6)
DIO	GPIO 23 (Pin 16)
CLK	GPIO 24 (Pin 18)

STEPS:

Step 1: Connect TM1637 to Raspberry Pi

Connect the TM1637 display to Raspberry Pi according to the following connections:

VCC → 5V
GND → GND
DIO → GPIO 23
CLK → GPIO 24

Step 2: Open Raspberry Pi Terminal

Open the **Terminal** in Raspberry Pi.

Step 3: Install Python Library

Install the TM1637 Python library using:

```
pip3 install tm1637
```

If required, install GPIO support:

```
pip3 install RPi.GPIO
```

Step 4: Create Python Program

Create a Python file:

```
nano clock.py
```

Step 5: Write the Program

```
import sys
import time
import datetime
import RPi.GPIO as GPIO
import tm1637
Display = tm1637.TM1637(23, 24, 5)
# Display.Clear()
Display.brightness(None)
while(True):
    now = datetime.datetime.now()
    hour = now.hour
    minute = now.minute
    second = now.second
    hour, minute = datetime.datetime.now().hour, datetime.datetime.now().minute
    # Display.show(f"{hour}{minute}")
    Display.show(f"{minute}{second}")
```

```
print(f"{minute}{second}")
# print(f"{hour}{minute}")
# Display.showDoublepoint({second:2})
time.sleep(0.25)
```

Step 6: Save the Program

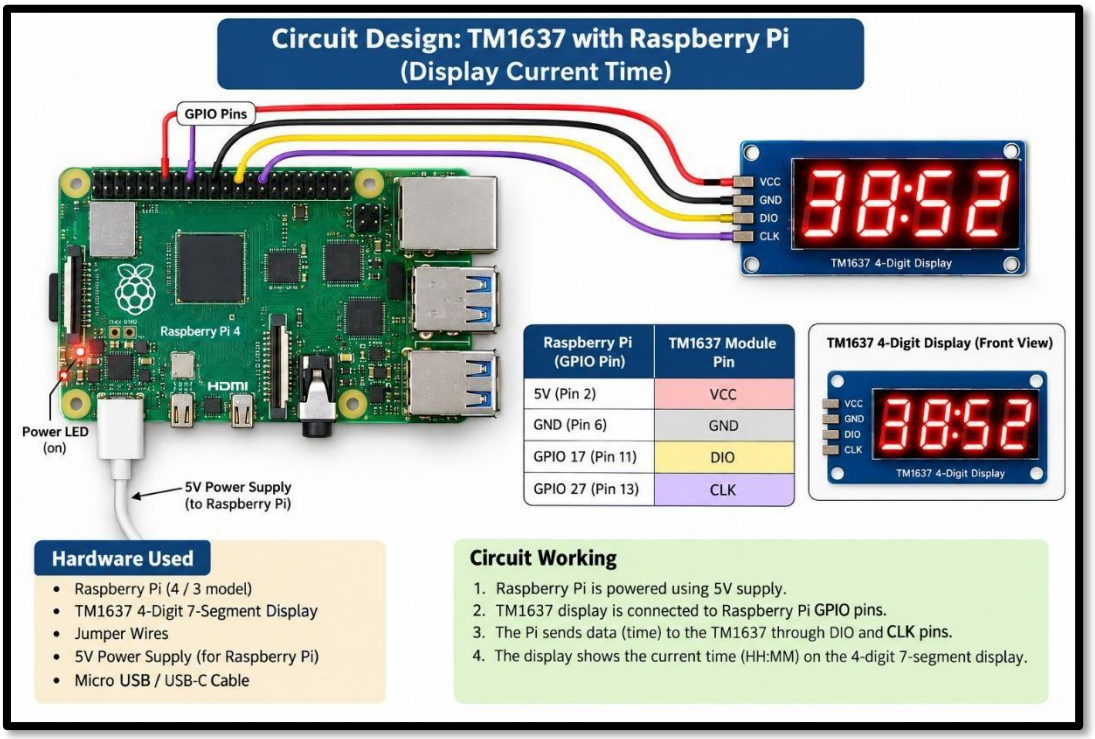
In nano:

- Press **Ctrl + X**
- Press **Y**
- Press **Enter**

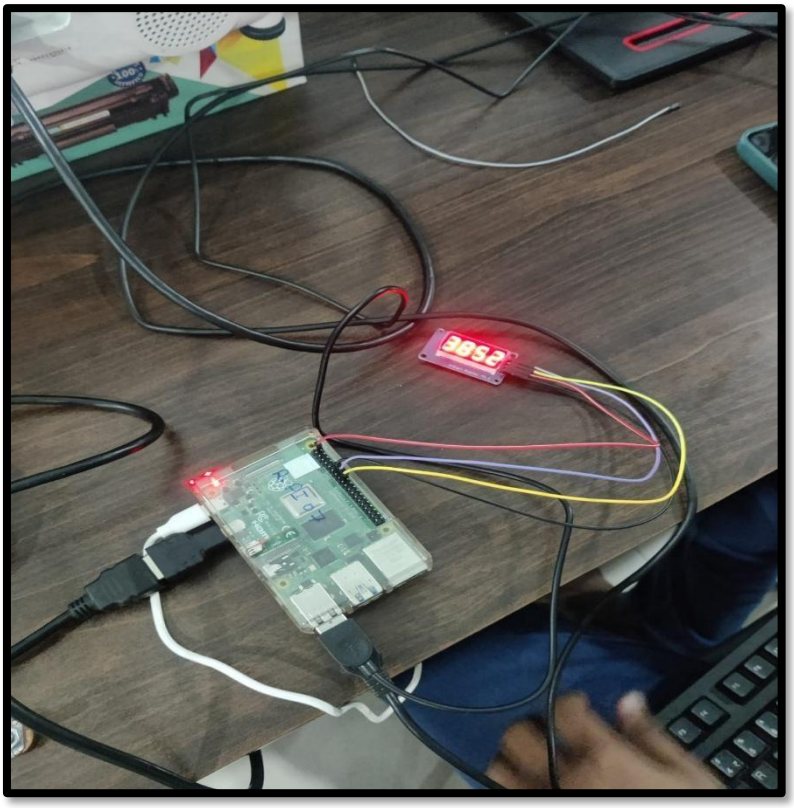
Step 7: Run the Program

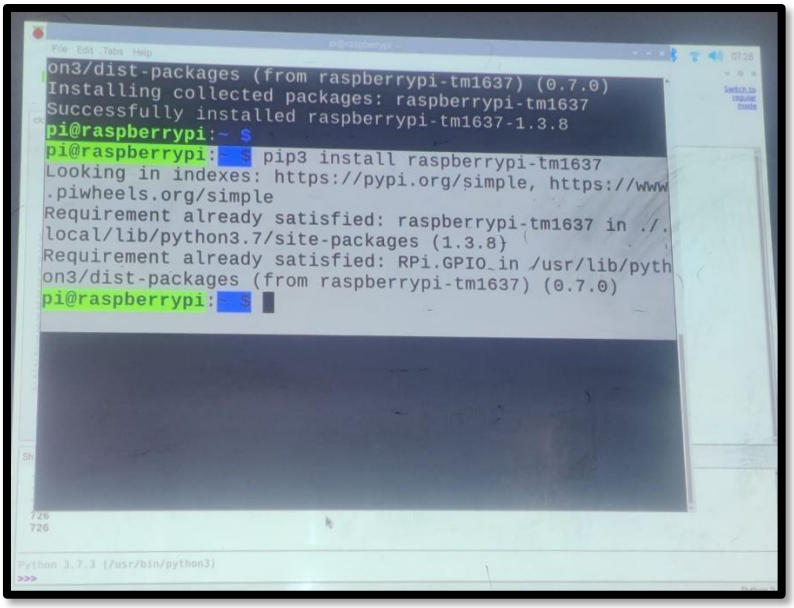
```
python3 clock.py
```

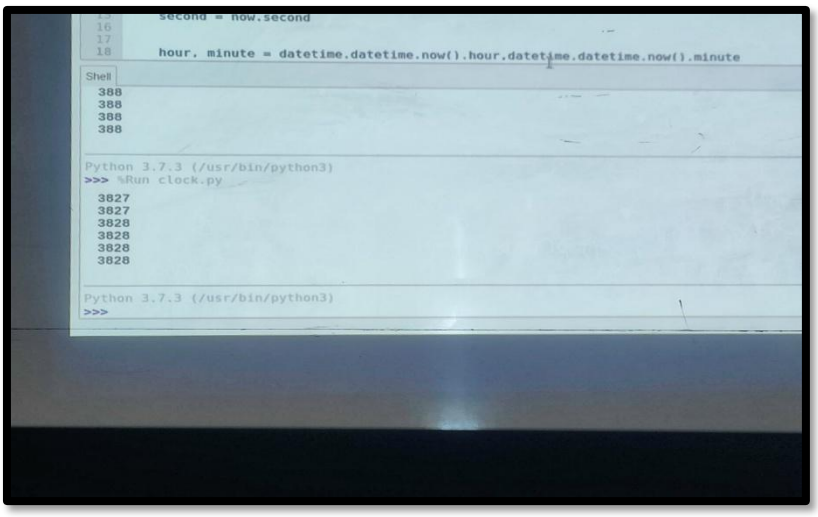
CIRCUIT DIAGRAM:



OUTPUT:




on3/dist-packages (from raspberrypi-tm1637) (0.7.0)
Installing collected packages: raspberrypi-tm1637
Successfully installed raspberrypi-tm1637-1.3.8
pi@raspberrypi:~ $
pi@raspberrypi:~ $ pip3 install raspberrypi-tm1637
Looking in indexes: https://pypi.org/simple, https://www
.piwheels.org/simple
Requirement already satisfied: raspberrypi-tm1637 in ./.
local/lib/python3.7/site-packages (1.3.8)
Requirement already satisfied: RPi.GPIO in /usr/lib/pyth
on3/dist-packages (from raspberrypi-tm1637) (0.7.0)
pi@raspberrypi:~ $


second = now.second
hour, minute = datetime.datetime.now().hour,datetime.datetime.now().minute
Shell
388
388
388
388
Python 3.7.3 (/usr/bin/python3)
>>> %Run clock.py
3827
3827
3828
3828
3828
3828
Python 3.7.3 (/usr/bin/python3)
>>>

PRACTICAL 4

AIM: INTERFACE GPS MODULE WITH RASPBERRY PI

HARDWARE REQUIREMENTS

1. Raspberry Pi
2. GPS Module (NEO-6M)
3. Jumper wires
4. Power supply
5. MicroSD card with Raspberry Pi OS

SOFTWARE REQUIREMENTS

1. Raspberry Pi OS
2. Python 3
3. pyserial Python library

GPS MODULE PIN CONFIGURATION

For a typical **NEO-6M GPS module**:

GPS Pin	Raspberry Pi
VCC	5V
GND	GND
TX	GPIO 15 (RXD)
RX	GPIO 14 (TXD)

PROCEDURE

Step 1: Connect the GPS Module

Connect the GPS module to Raspberry Pi as follows:

- **VCC → 5V**
- **GND → GND**

- **TX → GPIO 15 (RXD)**
- **RX → GPIO 14 (TXD)**

Step 2: Enable Serial Interface

Open Raspberry Pi Terminal and enter:

sudo raspi-config

Go to:

Interface Options → Serial Port

Select:

- Login shell over serial → **No**
- Serial port hardware → **Yes**

Then reboot:

```
sudo reboot
```

Step 3: Install PySerial

After reboot, open Terminal and enter:

```
pip3 install pyserial
```

Step 4: Create Python Program

Open Thonny Python IDE and create a new file:

gps.py

Step 5: Write the Python Program

```
import serial
import time
# Open serial port
gps = serial.Serial('/dev/serial0', baudrate=9600, timeout=1)
```

```
while True:
    data = gps.readline().decode('ascii', errors='replace')
    if data.startswith('$GPGGA'):
        print(data)
    time.sleep(1)
```

Step 6: Run the Program

Click **Run** in Thonny.

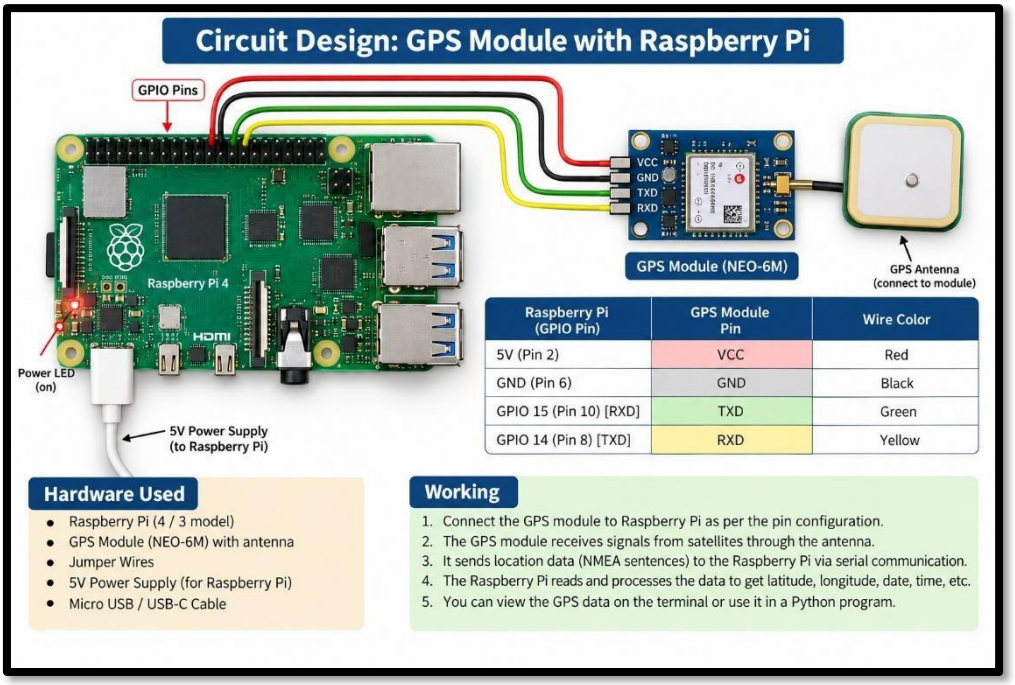
The GPS module will start sending NMEA GPS data to the Raspberry Pi.

Step 7: Observe GPS Data

The GPS module can provide information such as:

- Latitude
- Longitude
- Altitude
- Number of satellites
- GPS time
- Position fix

CIRCUIT DIAGRAM:



OUTPUT:

PRACTICAL 5

AIM: INTERFACE ADS1115 WITH RASPBERRY PI TO ACT AS AN OSCILLOSCOPE

HARDWARE REQUIREMENTS

1. Raspberry Pi
2. ADS1115 16-bit ADC module
3. Jumper wires
4. Signal source / potentiometer / function generator
5. Power supply

SOFTWARE REQUIREMENTS

1. Raspberry Pi OS
2. Python 3
3. Adafruit ADS1x15 library
4. Matplotlib library

ADS1115 PIN CONNECTIONS

ADS1115 Pin	Raspberry Pi Pin
VDD	3.3V — Pin 1
GND	GND — Pin 6
SDA	GPIO 2 / SDA — Pin 3
SCL	GPIO 3 / SCL — Pin 5
ADDR	GND
A0	Input signal

PROCEDURE

Step 1: Connect ADS1115 to Raspberry Pi

Connect the ADS1115 module using the I²C interface:

- VDD → 3.3V
- GND → GND
- SDA → GPIO 2
- SCL → GPIO 3
- ADDR → GND
- A0 → Input signal

Step 2: Enable I²C

Open the Raspberry Pi Terminal:

```
sudo raspi-config
```

Select:

Interface Options → I2C → Enable

Then restart:

```
sudo reboot
```

Step 3: Check ADS1115 Connection

After reboot, enter:

```
sudo apt install -y i2c-tools
```

Then:

```
i2cdetect -y 1
```

The ADS1115 normally appears at address:

```
48
```

Step 4: Install Required Libraries

Install the ADS1115 library:

```
pip3 install adafruit-circuitpython-ads1x15
```

Install Matplotlib:

pip3 install matplotlib

Step 5: Create Python Program

Open **Thonny Python IDE** and create:

oscilloscope.py

Enter the following program:

```
import time
import board
import busio
import matplotlib.pyplot as plt
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
# Initialize I2C
i2c = busio.I2C(board.SCL, board.SDA)
# Initialize ADS1115
ads = ADS.ADS1115(i2c)
# Select channel A0
chan = AnalogIn(ads, ADS.P0)
# Lists for waveform
time_data = []
voltage_data = []
start_time = time.time()
plt.ion()
fig, ax = plt.subplots()
while True:
    voltage = chan.voltage
    current_time = time.time() - start_time
```

```
    time_data.append(current_time)
    voltage_data.append(voltage)
    # Keep recent samples
    if len(time_data) > 200:
        time_data.pop(0)
        voltage_data.pop(0)
    ax.clear()
    ax.plot(time_data, voltage_data)
    ax.set_xlabel("Time (seconds)")
    ax.set_ylabel("Voltage (V)")
    ax.set_title("ADS1115 Oscilloscope")
    ax.grid(True)
    plt.pause(0.01)
```

Step 6: Run the Program

Click the **Run** button in Thonny.

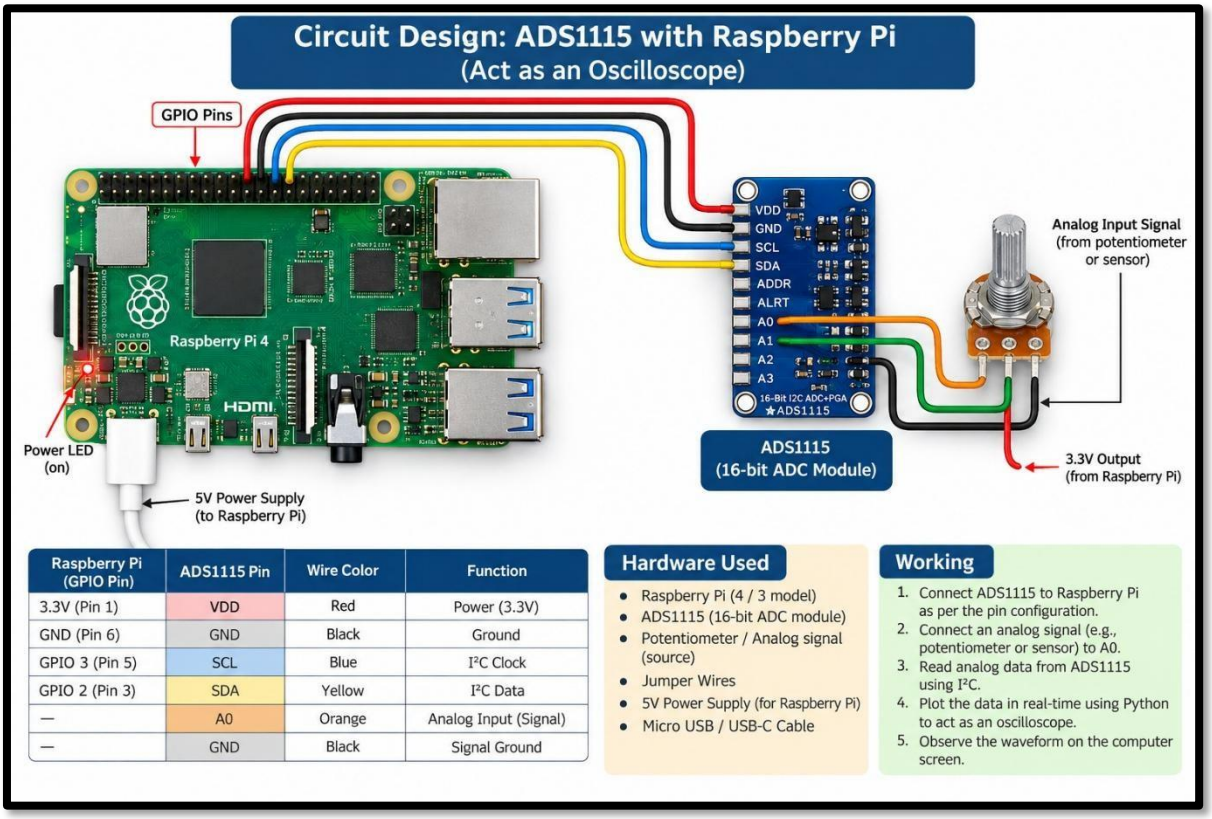
The ADS1115 continuously reads the analog voltage applied to **A0**.

The program plots:

- **X-axis → Time**
- **Y-axis → Voltage**

Thus, the waveform can be observed like a basic oscilloscope.

CIRCUIT DIAGRAM:



Raspberry Pi (GPIO Pin)	ADS1115 Pin	Wire Color	Function
3.3V (Pin 1)	VDD	Red	Power (3.3V)
GND (Pin 6)	GND	Black	Ground
GPIO 3 (Pin 5)	SCL	Blue	I²C Clock
GPIO 2 (Pin 3)	SDA	Yellow	I²C Data
—	A0	Orange	Analog Input (Signal)
—	GND	Black	Signal Ground

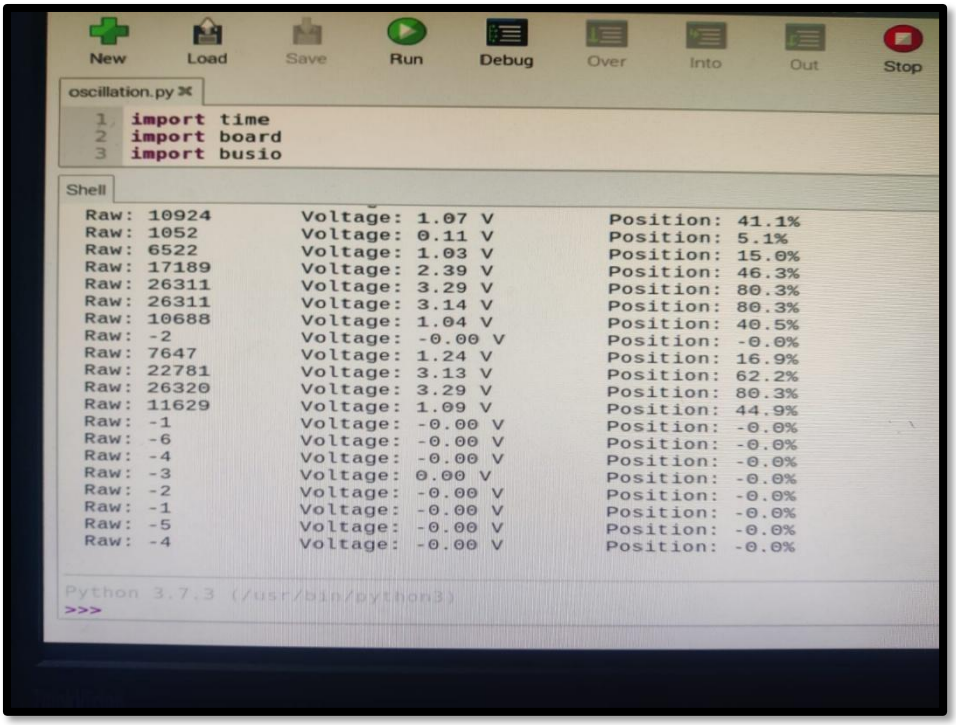
Hardware Used

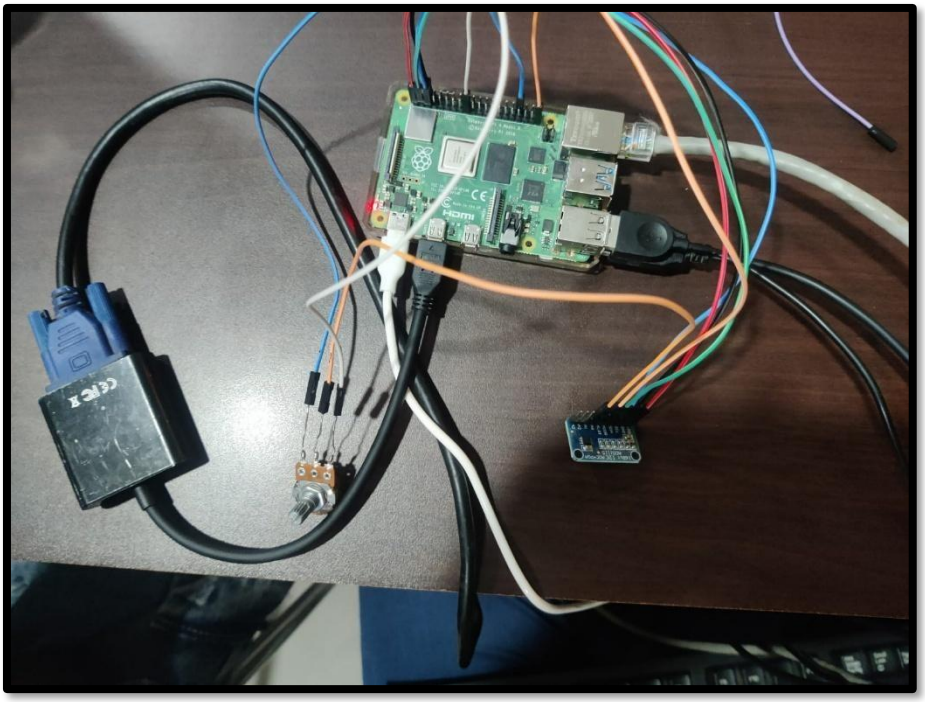
- Raspberry Pi (4 / 3 model)
- ADS1115 (16-bit ADC module)
- Potentiometer / Analog signal (source)
- Jumper Wires
- 5V Power Supply (for Raspberry Pi)
- Micro USB / USB-C Cable

Working

1. Connect ADS1115 to Raspberry Pi as per the pin configuration.
2. Connect an analog signal (e.g., potentiometer or sensor) to A0.
3. Read analog data from ADS1115 using I²C.
4. Plot the data in real-time using Python to act as an oscilloscope.
5. Observe the waveform on the computer screen.

OUTPUT:





PRACTICAL 6

AIM: INTERFACE 16X2 LCD WITH RASPBERRY PI USING I2C MODULE TO DISPLAY TEXT.

Hardware Requirements

1. Raspberry Pi
2. 16×2 LCD Display
3. I²C LCD Module (PCF8574)
4. Jumper wires
5. Power supply

Software Requirements

1. Raspberry Pi OS
2. Python 3
3. smbus / smbus2 Python library

Pin Connections

The I²C module reduces the LCD connections to four main pins.

I²C Module	Raspberry Pi
VCC	5V — Pin 2
GND	GND — Pin 6
SDA	GPIO 2 / SDA — Pin 3
SCL	GPIO 3 / SCL — Pin 5

The I²C module is connected to the back of the 16×2 LCD.

Procedure

Step 1: Connect the LCD

Connect the I²C LCD module to the Raspberry Pi:

- **VCC → 5V**
- **GND → GND**
- **SDA → GPIO 2**
- **SCL → GPIO 3**

Step 2: Enable I²C

Open the Raspberry Pi Terminal and type:

```
sudo raspi-config
```

Select:

Interface Options → I2C → Enable

Then restart Raspberry Pi:

```
sudo reboot
```

Step 3: Install I²C Tools

Open Terminal and enter:

```
sudo apt install -y i2c-tools
```

Step 4: Find the LCD I²C Address

Run:

```
i2cdetect -y 1
```

A common LCD address is:

```
27
```

Some modules use:

```
3F
```

Use the address shown by your Raspberry Pi, rather than assuming 0x27.

Step 5: Install Python Library

Install the SMBus library:

```
pip3 install smbus2
```

Step 6: Create Python Program

Open **Thonny Python IDE** and create a new file:

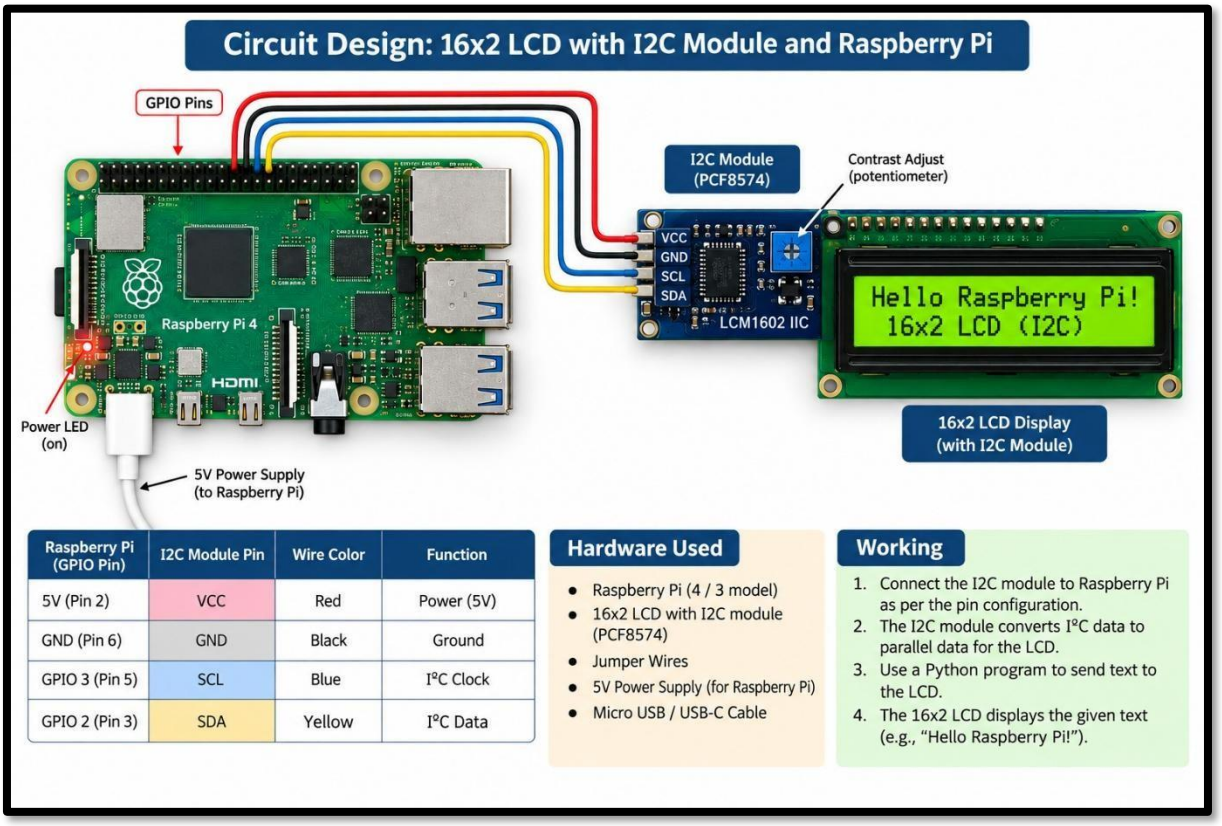
lcd.py

Step 7: Enter the Program

```
import smbus2
import time
LCD_ADDR = 0x27
bus = smbus2.SMBus(1)
LCD_CHR = 1
LCD_CMD = 0
LCD_LINE_1 = 0x80
LCD_LINE_2 = 0xC0
ENABLE = 0b00000100
BACKLIGHT = 0b00001000
def lcd_write(data, mode):
    high = data & 0xF0
    low = (data << 4) & 0xF0
    bus.write_byte(LCD_ADDR, high | mode | BACKLIGHT)
    bus.write_byte(LCD_ADDR, high | mode | ENABLE | BACKLIGHT)
    time.sleep(0.0005)
    bus.write_byte(LCD_ADDR, high | mode | BACKLIGHT)
    bus.write_byte(LCD_ADDR, low | mode | BACKLIGHT)
    bus.write_byte(LCD_ADDR, low | mode | ENABLE | BACKLIGHT)
    time.sleep(0.0005)
    bus.write_byte(LCD_ADDR, low | mode | BACKLIGHT)
def lcd_init():
    lcd_write(0x33,  LCD_CMD)
    lcd_write(0x32,  LCD_CMD)
    lcd_write(0x06,  LCD_CMD)
    lcd_write(0x0C, LCD_CMD)
```

```
    lcd_write(0x28, LCD_CMD)
    lcd_write(0x01, LCD_CMD)
    time.sleep(0.002)
def lcd_display(text, line):
    lcd_write(line, LCD_CMD)
    for char in text:
        lcd_write(ord(char), LCD_CHR)
lcd_init()
lcd_display("Hello Raspberry", LCD_LINE_1)
lcd_display("IoT Practical", LCD_LINE_2)
time.sleep(10)
```

CIRCUIT DIAGRAM:



Raspberry Pi (GPIO Pin)	I2C Module Pin	Wire Color	Function
5V (Pin 2)	VCC	Red	Power (5V)
GND (Pin 6)	GND	Black	Ground
GPIO 3 (Pin 5)	SCL	Blue	I²C Clock
GPIO 2 (Pin 3)	SDA	Yellow	I²C Data

OUTPUT:

PRACTICAL 7

AIM: INTERFACE RFID MODULE WITH RASPBERRY PI TO READ AND WRITE RFID TAGS/CARDS

Hardware Requirements

1. Raspberry Pi
2. RC522 RFID Reader Module
3. RFID Card/Tag
4. Jumper wires
5. Power supply

Software Requirements

1. Raspberry Pi OS
2. Python 3
3. MFRC522 Python library
4. SPI interface

RC522 to Raspberry Pi Connections

The RC522 communicates with Raspberry Pi using **SPI communication**.

RC522 Pin	Raspberry Pi Pin
3.3V	3.3V — Pin 1
GND	GND — Pin 6
SDA / SS	GPIO 8 / CE0 — Pin 24
SCK	GPIO 11 / SCLK — Pin 23
MOSI	GPIO 10 / MOSI — Pin 19
MISO	GPIO 9 / MISO — Pin 21
IRQ	Not connected
RST	GPIO 25 — Pin 22

Procedure

Step 1: Connect the RC522 RFID Module

Connect the RC522 to Raspberry Pi according to the table above.

The important SPI connections are:

- **SDA/SS → GPIO 8**
- **SCK → GPIO 11**
- **MOSI → GPIO 10**
- **MISO → GPIO 9**
- **RST → GPIO 25**
- **3.3V → 3.3V**
- **GND → GND**

Step 2: Enable SPI

Open the Raspberry Pi Terminal:

sudo raspi-config

Go to:

Interface Options → SPI → Enable

Then restart Raspberry Pi:

sudo reboot

Step 3: Install Required Library

After reboot, open Terminal and install the RFID library:

pip3 install mfrc522

If required, install GPIO support:

pip3 install RPi.GPIO

Step 4: Create Python Program

Open **Thonny Python IDE** and create a new file:

rfid.py

Step 5: Program to Read RFID Card

Enter the following program:

```
from mfrc522 import SimpleMFRC522
reader = SimpleMFRC522()
print("Place your RFID card/tag near the reader...")
try:
    id, text = reader.read()
    print("Card ID:", id)
    print("Data:", text)
finally:
    import RPi.GPIO as GPIO
    GPIO.cleanup()
```

Step 6: Run the Program

Run the program in Thonny.

Place the RFID card/tag near the **RC522 reader**.

The card ID and stored data will be displayed in the Thonny Shell.

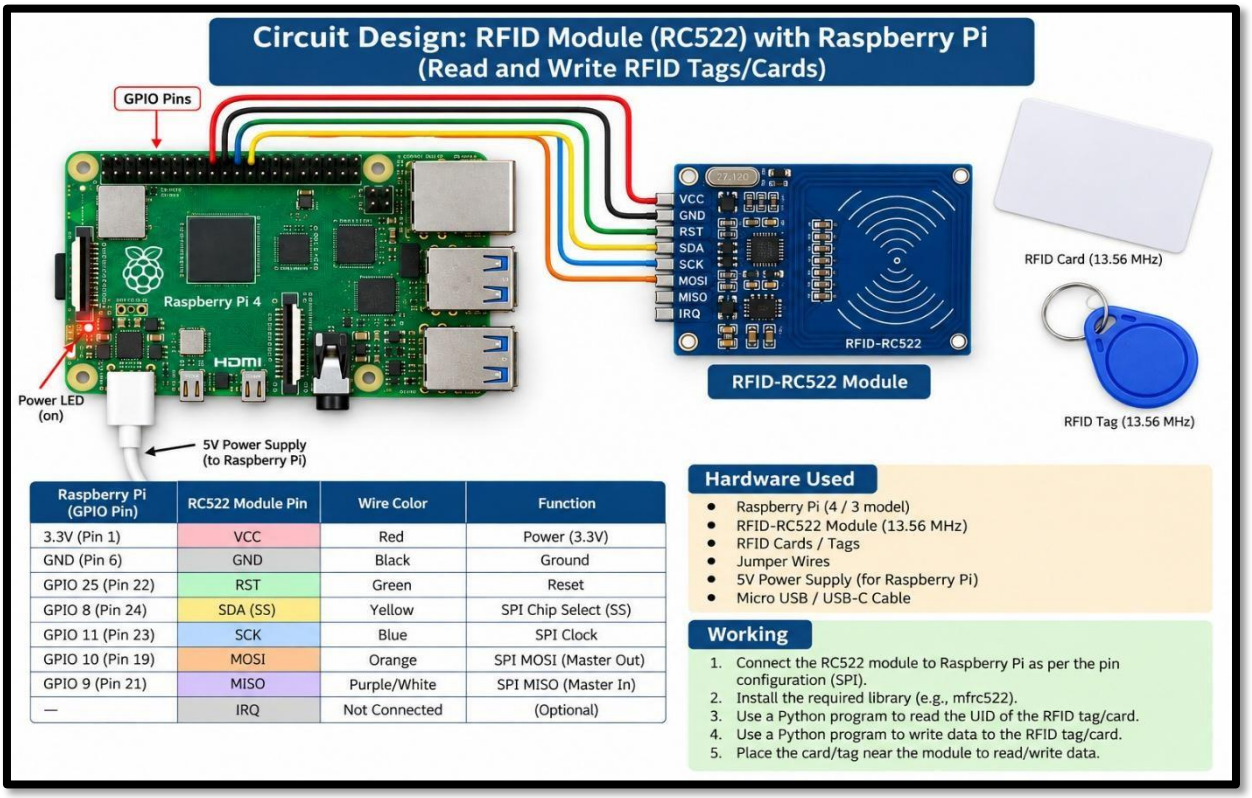
Expected Output

Place your RFID card/tag near the reader...

Card ID: 123456789

Data: Hello Raspberry Pi

CIRCUIT DIAGRAM:



Raspberry Pi (GPIO Pin)	RC522 Module Pin	Wire Color	Function
3.3V (Pin 1)	VCC	Red	Power (3.3V)
GND (Pin 6)	GND	Black	Ground
GPIO 25 (Pin 22)	RST	Green	Reset
GPIO 8 (Pin 24)	SDA (SS)	Yellow	SPI Chip Select (SS)
GPIO 11 (Pin 23)	SCK	Blue	SPI Clock
GPIO 10 (Pin 19)	MOSI	Orange	SPI MOSI (Master Out)
GPIO 9 (Pin 21)	MISO	Purple/White	SPI MISO (Master In)
—	IRQ	Not Connected	(Optional)

Hardware Used

- Raspberry Pi (4 / 3 model)
- RFID-RC522 Module (13.56 MHz)
- RFID Cards / Tags
- Jumper Wires
- 5V Power Supply (for Raspberry Pi)
- Micro USB / USB-C Cable

Working

1. Connect the RC522 module to Raspberry Pi as per the pin configuration (SPI).
2. Install the required library (e.g., mfrc522).
3. Use a Python program to read the UID of the RFID tag/card.
4. Use a Python program to write data to the RFID tag/card.
5. Place the card/tag near the module to read/write data.

OUTPUT:

PRACTICAL 8

AIM: INTERFACE FINGERPRINT SENSOR WITH RASPBERRY PI

Hardware Requirements

1. Raspberry Pi
2. Fingerprint Sensor Module (R307 / AS608)
3. Jumper wires
4. Power supply

Software Requirements

1. Raspberry Pi OS
2. Python 3
3. PySerial library

Fingerprint Sensor Pin Configuration

For a typical **R307/AS608 fingerprint sensor**:

Sensor Pin	Raspberry Pi
VCC	5V
GND	GND
TX	GPIO 15 / RXD
RX	GPIO 14 / TXD

Procedure

Step 1: Connect the Fingerprint Sensor

Connect the sensor to Raspberry Pi:

- **VCC → 5V**
- **GND → GND**
- **TX → GPIO 15 (RXD)**
- **RX → GPIO 14 (TXD)**

Step 2: Enable Serial Communication

Open the Raspberry Pi Terminal:

```
sudo raspi-config
```

Select:

Interface Options → Serial Port

Set:

- Login shell over serial → **No**
- Serial port hardware → **Yes**

Restart Raspberry Pi:

```
sudo reboot
```

Step 3: Install PySerial

Open Terminal and install the Python serial library:

```
pip3 install pyserial
```

Step 4: Create Python Program

Open **Thonny Python IDE** and create a new file:

```
fingerprint.py
```

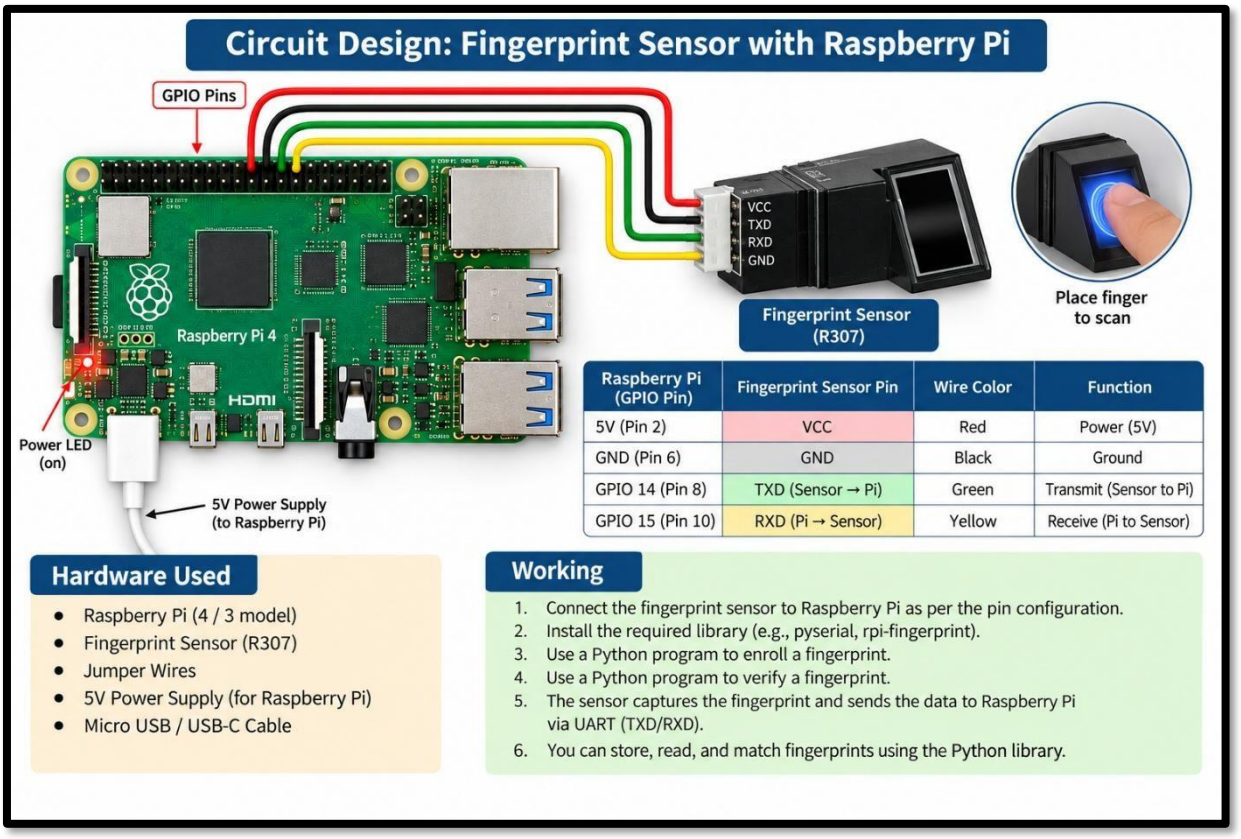
Step 5: Test Serial Communication

Enter the following program:

```
import serial
import time
sensor = serial.Serial(
    port='/dev/serial0',
    baudrate=57600,
    timeout=1
)
print("Fingerprint sensor connected.")
```

```
while True:
    data = sensor.read(32)
    if data:
        print("Data received:", data.hex())
    time.sleep(1)
```

CIRCUIT DIAGRAM:



OUTPUT:

PRACTICAL 9

AIM: INTERFACE PI CAMERA WITH RASPBERRY PI TO CAPTURE IMAGE AND VIDEO.

Hardware Requirements

1. Raspberry Pi
2. Raspberry Pi Camera Module
3. Camera ribbon cable
4. MicroSD card with Raspberry Pi OS
5. Power supply

Software Requirements

1. Raspberry Pi OS
2. Python 3
3. Picamera2 library

Procedure

Step 1: Connect the Pi Camera

1. Switch **OFF** the Raspberry Pi.
2. Locate the **CSI camera connector** on the Raspberry Pi.
3. Insert the camera ribbon cable into the CSI connector.
4. Make sure the cable is properly aligned and firmly connected.
5. Connect the other end of the cable to the camera module.
6. Switch ON the Raspberry Pi.

Step 2: Enable and Check the Camera

Open the Terminal and check whether the camera is detected:

```
rpicam-hello
```

A preview window should appear if the camera is working correctly.

On newer Raspberry Pi OS versions, **rpicam-* commands** are used instead of the older raspistill and raspivid commands.

Step 3: Capture an Image

Use the following command:

```
rpicam-still -o image.jpg
```

The camera captures an image and saves it as:

```
image.jpg
```

To specify a capture delay:

```
rpicam-still -t 5000 -o image.jpg
```

Here, 5000 means **5000 milliseconds (5 seconds)**.

Step 4: Capture Video

To record a video for 10 seconds:

```
rpicam-vid -t 10000 -o video.h264
```

Here:

- 10000 = 10 seconds
- video.h264 = output video file

The video will be saved in the current directory.

Step 5: Capture Image Using Python

Create a Python file:

```
nano camera.py
```

Enter the following program:

```
from picamera2 import Picamera2
import time
picam2 = Picamera2()
picam2.start()
time.sleep(2)
picam2.capture_file("image.jpg")
print("Image captured successfully.")
picam2.stop()
```

Save the file and run:

```
python3 camera.py
```

Expected Output

Image captured successfully.

An image named:

image.jpg

will be created.

Step 6: Capture Video Using Python

Create another Python file:

```
nano video.py
```

Enter:

```
from picamera2 import Picamera2
import time
picam2 = Picamera2()
picam2.start()
time.sleep(2)
picam2.start_recording("video.h264")
print("Recording video...")
time.sleep(10)
picam2.stop_recording()
print("Video captured successfully.")
picam2.stop()
```

Run the program:

```
python3 video.py
```

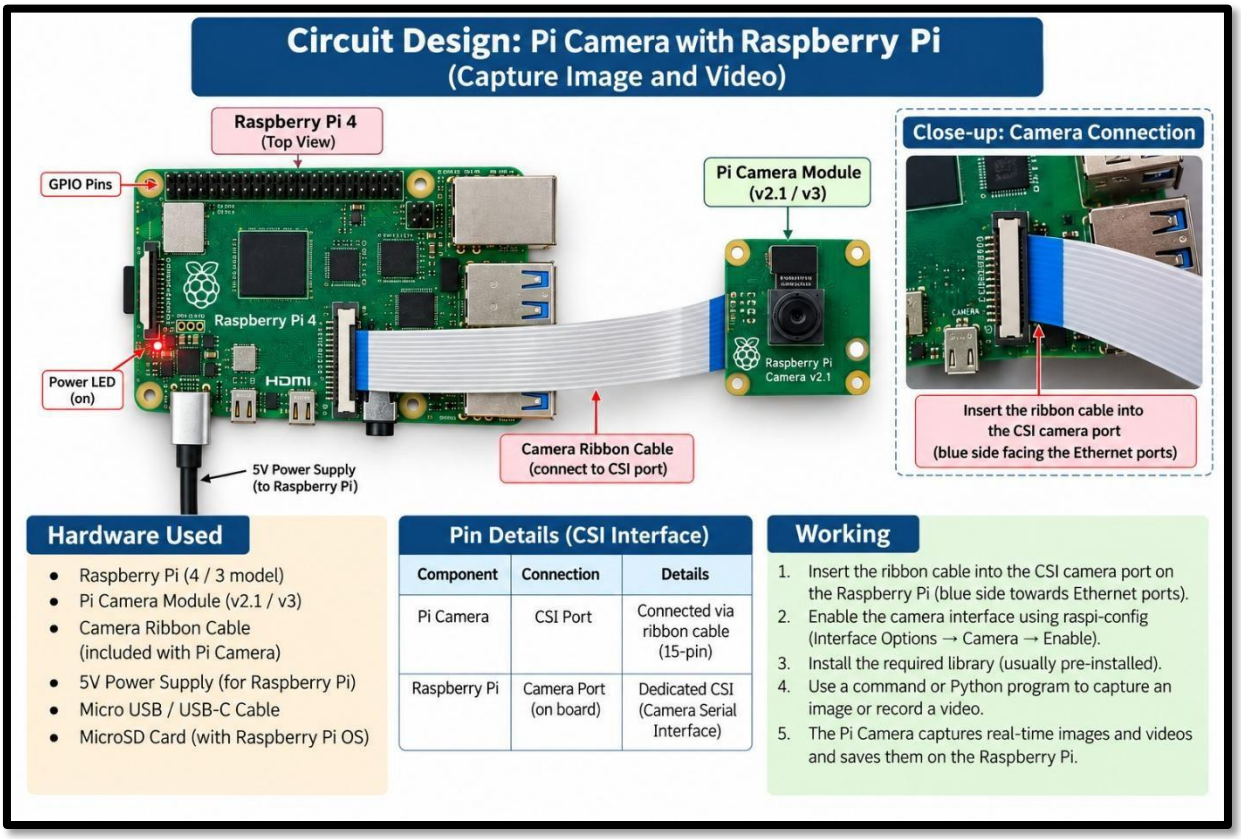
The camera will record video for **10 seconds**.

Expected Output

Recording video...

Video captured successfully.

CIRCUIT DIAGRAM:



OUTPUT:

PRACTICAL 10

AIM: DESIGN IOT-BASED HOME AUTOMATION SYSTEM USING RASPBERRY PI

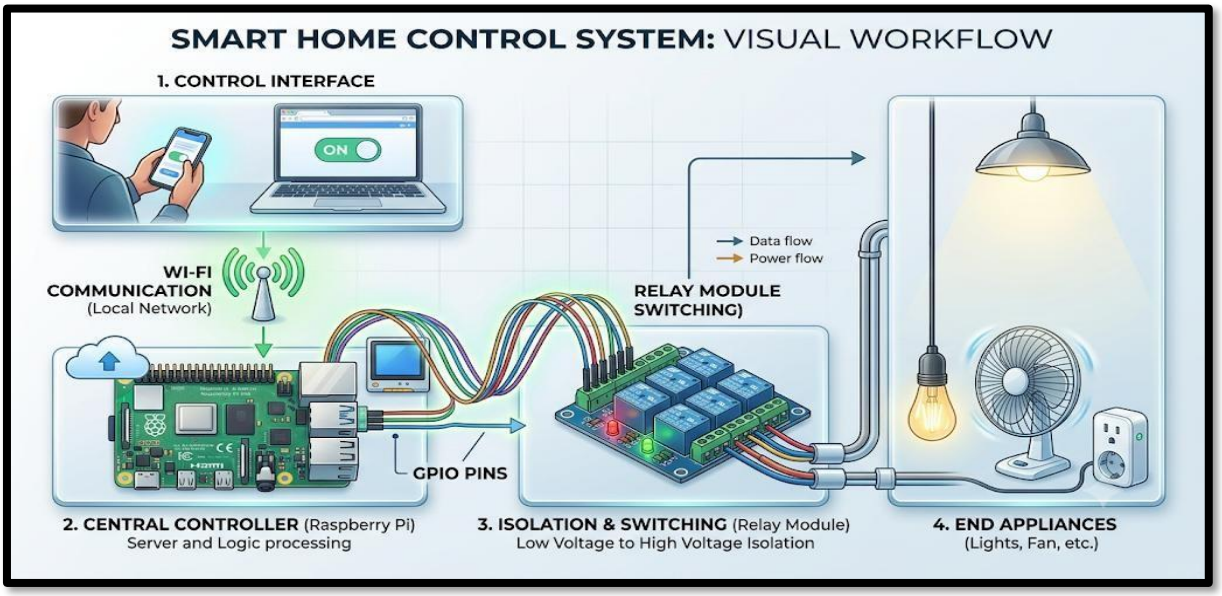
Hardware Requirements

1. Raspberry Pi
2. 1-Channel or 2-Channel Relay Module
3. LED / Bulb
4. Jumper wires
5. Breadboard
6. Resistors
7. Power supply
8. Wi-Fi connection

Software Requirements

1. Raspberry Pi OS
2. Python 3
3. GPIO library
4. Web browser

Block Diagram



Circuit Connections

For a simple demonstration, connect one relay module to an LED.

Relay Module	Raspberry Pi
VCC	5V
GND	GND
IN	GPIO 18

Procedure

Step 1: Connect Raspberry Pi

Connect the Raspberry Pi to the power supply and make sure it is connected to the **Wi-Fi network**.

Step 2: Connect Relay Module

Connect the relay module to Raspberry Pi:

- **VCC → 5V**
- **GND → GND**
- **IN → GPIO 18**

Connect the demonstration appliance/load to the relay output.

Step 3: Open Terminal

Open the Raspberry Pi Terminal.

Create a Python program:

```
nano home_automation.py
```

Step 4: Write the Python Program

```
import RPi.GPIO as GPIO
import time
RELAY = 18
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY, GPIO.OUT)
```

```
try:
    while True:
        print("1. Turn ON")
        print("2. Turn OFF")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            GPIO.output(RELAY, GPIO.HIGH)
            print("Appliance ON")
        elif choice == "2":
            GPIO.output(RELAY, GPIO.LOW)
            print("Appliance OFF")
        elif choice == "3":
            break
        else:
            print("Invalid choice")
finally:
    GPIO.output(RELAY, GPIO.LOW)
    GPIO.cleanup()
```

Step 5: Run the Program

Save the program and run:

python3 home_automation.py

The following menu will appear:

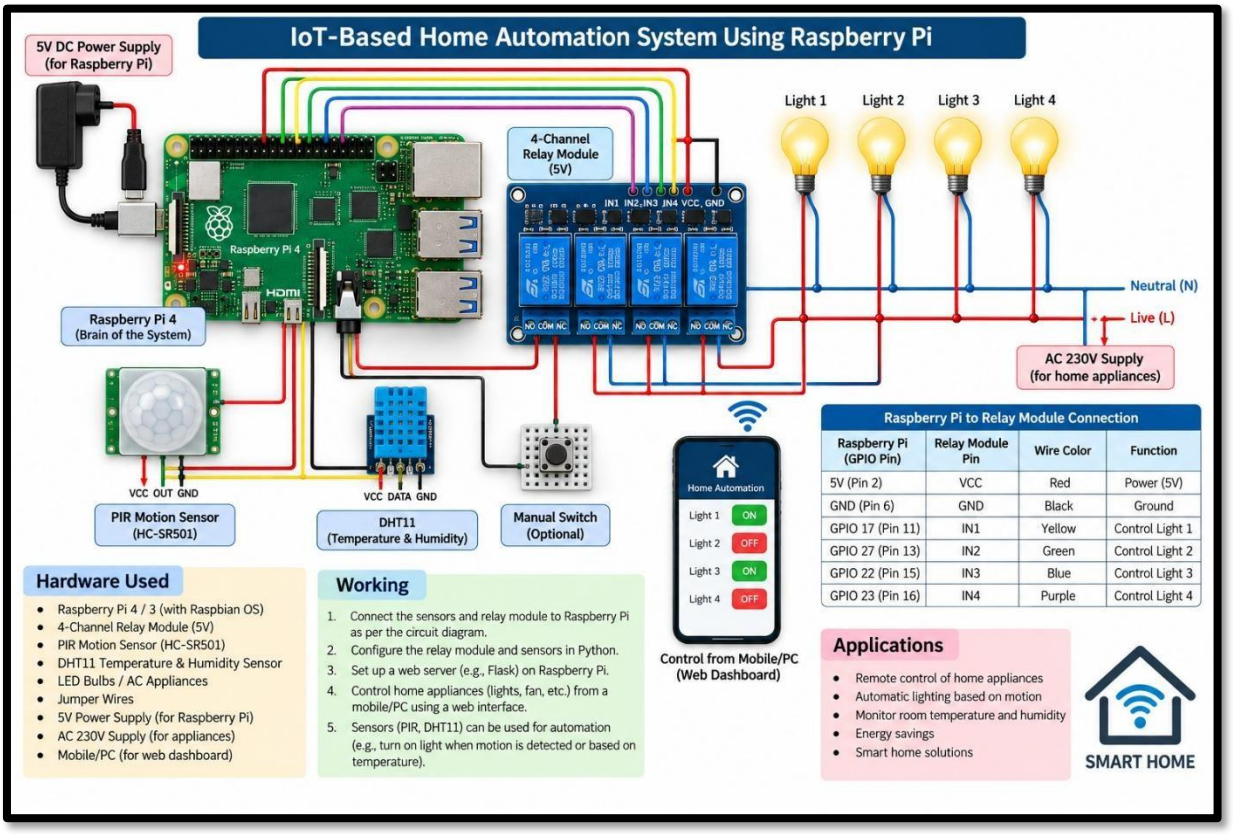
1. Turn ON

2. Turn OFF

3. Exit

Enter your choice:

CIRCUIT DIAGRAM:



OUTPUT: